

SENAI – ESCOLA HERMENEGILDO CAMPOS DE ALMEIDA CURSO
TÉCNICO EM CIBERSISTEMAS PARA AUTOMAÇÃO – 3º SEMESTRE

GILBERTO ALVESDE MELO RAMOS
JÚLIA LOPES DA SILVA
PEDRO ARTHUR BARBANCHO SANTOS

DOCUMENTAÇÃO TÉCNICA
JGP CORPORATION–PROJETO INTEGRADOR

GUARULHOS – SP
2026

GILBERTO ALVESDE MELO RAMOS
JÚLIA LOPES DA SILVA
PEDRO ARTHUR BARBANCHO SANTOS

DOCUMENTAÇÃO TÉCNICA
JGP CORPORATION – PROJETO INTEGRADOR

Documentação apresentada como uma junção de todas as disciplinas aplicadas a um só projeto integrador do curso técnico em Cbersistemas para automação como requisito parcial para aprovação semestral, sob a orientação do prof. William Belarmino da Silva.

GUARULHOS-SP
2026

SUMÁRIO

3.1 CONTEXTUALIZAÇÃO DO PROJETO	06
3.1.1 Implementação da API	06
3.1.2 Rotas da API	06
3.2 OBJETIVO DA ETAPA 3	06
3.3 ARQUITETURA DO SISTEMA	07
3.3.1 Estrutura Geral	07
3.3.2 Elementos do Fluxograma	07
3.3.3 Comunicação TCP/IP	08
3.4 PESQUISA	09
3.4.1 O que é API REST	09
3.4.2 Métodos HTTP	09
3.4.3 JSON	09
3.4.4 Comunicação cliente/servidor	09
3.4.5 Rotas e endpoints	09
3.5 PROCESSAMENTO E VALIDAÇÃO DOS DADOS	10
3.5.1 Dados válidos e organizados	10
3.5.2 Verificação de campos obrigatórios	11
3.5.3 Tratamento de dados inválidos	11
3.5.4 Organização das informações	11
3.5.5 Registro de erros	12
3.5.6 Perguntas	12
3.6 INTEGRAÇÃO COM BANCO DE DADOS	13
3.7 IMPLEMENTAÇÃO DA INTEGRAÇÃO COM GOOGLE SHEETS	15
3.7.1 Autenticação Definida	16
3.7.2 Perguntas	16
3.8 SEGURANÇA INICIAL DO SISTEMA	18
3.8.1 Pesquisa	18
3.8.1.1 Tokens simples	18
3.8.1.2 Validação de requisições	18
3.8.1.3 Controle básico de acesso	18
3.8.1.4 Organização segura das credenciais	18
3.8.2 Perguntas	19
CONCLUSÃO	20
REFERÊNCIAS	21

PARTE 3 DO PROJETO INTEGRADOR

DESENVOLVIMENTO DA API, INTEGRAÇÃO COM BANCO DE DADOS E GOOGLE SHEETS

3.1 CONTEXTUALIZAÇÃO DO PROJETO

Contextualização: Em resumo o projeto deixou de ser apenas um dispositivo isolado (ESP8266) e passou a ser um sistema integrado que processa, armazena e exibe informações na nuvem automaticamente.

● Objetivo Geral do Sistema

O objetivo geral deste sistema é desenvolver uma solução integrada de automação baseada em IoT, capaz de coletar dados por meio do ESP8266, transmitir essas informações via rede, processá-las por meio de uma API, validar sua integridade, armazená-las em um banco de dados e integrá-las a plataformas em nuvem, como o Google Sheets, garantindo organização, monitoramento, segurança e confiabilidade no gerenciamento das informações.

● Aplicação do ESP8266 no Sistema

O ESP8266 é o componente responsável por realizar a coleta e o envio de dados dentro do sistema IoT desenvolvido. Ele atua como um dispositivo embarcado com conectividade Wi-Fi, permitindo a comunicação direta com a API por meio da rede.

3.1.1 Implementação da API

A API foi desenvolvida utilizando a linguagem Python com o framework Flask, sendo responsável por receber os dados enviados pelo ESP8266, validar as informações e retornar respostas ao dispositivo.

A seguir, apresenta-se um trecho do código responsável pelo recebimento dos dados:

```
from flask import Flask, request, jsonify

app = Flask(__name__)

@app.route('/dados', methods=['POST'])
def receber_dados():
    data = request.json

    if "temperatura" not in data:
        return jsonify({"erro": "Campo temperatura ausente"}), 400

    return jsonify({"status": "dados recebidos com sucesso"}), 200

app.run(host='0.0.0.0', port=8000)
```

Esse endpoint recebe requisições do tipo POST contendo dados no formato JSON. Caso os dados estejam incompletos, a API retorna um erro. Caso contrário, retorna uma confirmação de sucesso.

3.1.2 Rotas da API

E colocar:

- **POST /dados** → Recebe dados do ESP8266
- **GET /status** → Verificar funcionamento da API

3.2 OBJETIVO DA ETAPA 3

A Etapa 3 do projeto tem como principal objetivo transformar o sistema desenvolvido anteriormente em uma plataforma integrada de automação e monitoramento de dados. Nesta fase, o foco está no desenvolvimento de uma API capaz de receber as informações enviadas pelo ESP8266, processar os dados e realizar a comunicação entre os diferentes componentes do sistema.

Outro objetivo importante é a integração com o banco de dados que utilizamos o Python porque tem uma acessibilidade melhor, permitindo que todas as leituras recebidas sejam armazenadas de forma organizada e segura. Dessa maneira, o sistema poderá registrar informações como temperatura, umidade, rotação, estados do sistema, alertas e logs para futuras consultas e análises.

A etapa também contempla a integração com o Google Sheets, possibilitando a sincronização automática das informações coletadas com planilhas online. Essa integração facilita a visualização dos dados em nuvem, o compartilhamento das informações e o acompanhamento em tempo real do funcionamento do sistema.

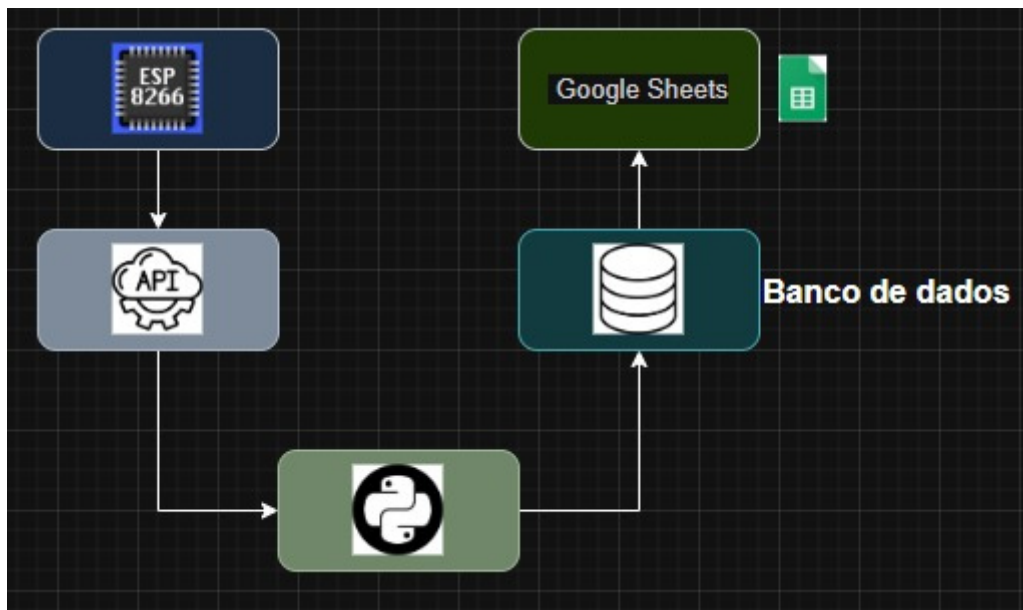
Além disso, a equipe deverá implementar o processamento automático dos dados, garantindo que as informações recebidas sejam verificadas, validadas e organizadas antes de serem armazenadas ou enviadas para outros serviços. Esse processo aumenta a confiabilidade do sistema e reduz erros durante a comunicação entre os dispositivos e as aplicações.

3.3 ARQUITETURA DO SISTEMA

3.3.1 Estrutura Geral

- Fluxo completo

ESP8266 → API → Python → Banco de Dados → Google Sheets



3.3.2 Elementos do Fluxograma

O sistema é composto pelos seguintes elementos:

- ESP8266 (coleta de dados)
- API (processamento e controle)
- Python (desenvolvimento da API, integração com o banco de dados e envio das informações para o Google Sheets)
- Banco de dados (armazenamento)
- Google Sheets (visualização)

Fluxo de funcionamento:

- O ESP8266 coleta dados (temperatura, rotação, etc.)
- Envia via TCP/IP em formato JSON para a API
- A API recebe e valida os dados
- Dados válidos são armazenados no banco
- Dados também são enviados ao Google Sheets
- Logs são registrados para monitoramento

3.3.3 Comunicação TCP/IP

- Como ocorre a comunicação: O Arduino utiliza a rede da escola para se comunicar com a aplicação em Python por meio do endereço IP da máquina que hospeda o servidor. No código, foi utilizado o IP 0.0.0,0 permitindo que o servidor aceitasse conexões da rede local.
- Endereço IP: 10.106.208.36
- Porta utilizada: Porta que está sendo utilizada é a 8.000.
- Protocolo HTTP : ESP8266 atua como cliente e o API atua como servidor

```
// --- PÁGINA WEB ---  
void paginaInicial() {  
    float t = dht.readTemperature();  
    String dh = obterDataHoraServidor();  
    String html = "<!DOCTYPE html><html><head><meta charset='UTF-8'><meta http-equiv='refresh' content='5'>";  
    html += "<style>body{background:#0f172a;color:white;font-family:sans-serif;text-align:center;}";  
    html += ".box{background:#1e293b;padding:20px;border-radius:15px;width:300px;margin:auto;margin-top:50px;}";  
    html += "h1{color:#38bdf8}.temp{font-size:40px;color:#22c55e;}</style></head><body>";  
    html += "<div class='box'><h1>🏠 Monitor IoT</h1><p>Local: " + localAparelho + "</p>";  
    html += "<p class='temp'>" + String(t) + "°C</p>";  
    html += "<p><b>Hora:</b> " + dh + "</p></div></body></html>";  
    server.send(200, "text/html", html);  
}
```

```
Conectando WiFi.....  
✅ WiFi Ativo!  
  
=====  
local: Laboratorio SENAI  
temperatura: 23.20 °C  
umidade: 55.00 %  
data / hora: 22/05/2026 11:56:38  
=====  
site: http://10.106.202.37  
=====
```

3.4 PESQUISA

- **O que é API REST :** Uma API REST é uma interface de programação de aplicativos (API) que segue os princípios de design do estilo arquitetônico REST (transferência de estado representacional), utilizado para conectar sistemas hipermídia distribuídos. As APIs REST também podem ser chamadas de APIs RESTful ou APIs web RESTful.
- **Métodos HTTP :** O protocolo HTTP define um conjunto de **métodos de requisição** responsáveis por indicar a ação a ser executada para um dado recurso. Embora esses métodos possam ser descritos como substantivos, eles também são comumente referenciados como ***HTTP Verbs (Verbos HTTP)***. Cada um deles implementa uma semântica diferente, mas alguns recursos são compartilhados por um grupo deles, como por exemplo, qualquer método de requisição pode ser do tipo safe, idempotent ou cacheable.
- **JSON :** Ou *JavaScript Object Notation*, é um formato leve e baseado em texto para armazenamento e intercâmbio de dados, amplamente utilizado em APIs web para troca de informações entre servidor e cliente. É legível por humanos e computadores, estruturado em pares de chave/valor, tornando-o um padrão eficiente.
- **Comunicação cliente/servidor :** A estrutura cliente-servidor é um modelo de arquitetura de rede amplamente utilizado em ambientes de TI. Nesta abordagem, os computadores são divididos em dois grupos: servidores, que fornecem serviços ou recursos, e clientes, que solicitam estes serviços e recursos, recebendo-os como respostas. A comunicação entre os dois é feita por meio de protocolos de rede, como TCP/IP, e é geralmente realizada pela internet ou por uma rede privada. Assim, o modelo é altamente escalável, flexível e seguro, o que permite às empresas distribuírem seus serviços e recursos em vários dispositivos e plataformas.
- **Rotas e endpoints :** Rotas e endpoints definem como uma API (Application Programming Interface) recebe solicitações e acessa recursos.

3.5 PROCESSAMENTO E VALIDAÇÃO DOS DADOS

3.5.1 Dados válidos e organizados

```
--- Enviando Dados ---
{
  "device": "ESP8266_M302",
  "timestamp_ms": 46754,
  "sensores": {
    "temperatura": 22.2,
    "umidade": 43,
    "rotacao": 50,
    "potenciometro": 507
  },
  "entradas": {
    "botao1": false,
    "botao2": false,
    "ir_recebido": false
  },
  "status": "normal",
  "wifi_rssi": -56
}
✅ Dados entregues a API com sucesso!
```

```
=====
🚀 API INICIADA! ESPERANDO ALGUM ARDUINO RESPONDER...
=====
```

```
❌ ALERTA: O Arduino no local 'Laboratorio SENAI'
⚠️ MOTIVO: O dispositivo não está respondendo!
🕒 Último contato foi às: 11:57:02
=====
```

```
=====
🔴 API FINALIZADA PELO USUÁRIO.
=====
```

3.5.2 Verificação de campos obrigatórios

O sistema realiza a validação dos campos obrigatórios enviados pelo ESP8266, verificando se todas as informações necessárias estão presentes no JSON recebido.

```
</> JSON  
  
{  
  "temperatura": 26.5,  
  "umidade": 70,  
  "rotacao": 1200,  
  "estado": "ativo"  
}
```

3.5.3 Tratamento de dados inválidos

A API também realiza a verificação dos tipos e valores recebidos para evitar inconsistências no sistema.

Exemplos de validação:

- Temperatura deve ser numérica
- Umidade deve estar entre 0% e 100%
- Rotação não pode possuir valores negativos
- Estado deve seguir valores previamente definidos

Quando um dado inválido é identificado:

- O sistema rejeita a requisição
- Um erro é registrado nos logs
- A API retorna uma resposta informando o problema encontrado

3.5.4 Organização das informações

Após a validação, os dados são organizados em estruturas padronizadas para facilitar:

- Armazenamento no banco de dados
- Consultas futuras
- Integração com Google Sheets
- Monitoramento do sistema

3.5.5 Registro de erros

```
--- Enviando Dados ---
{
  "device": "ESP8266_M302",
  "timestamp_ms": 577319,
  "sensores": {
    "temperatura": 23.7,
    "umidade": 55,
    "rotacao": 0,
    "potenciometro": 5
  },
  "entradas": {
    "botao1": false,
    "botao2": false,
    "ir_recebido": false
  },
  "status": "normal",
  "wifi_rssi": -56
}
✗ Falha no POST (A API está ligada?): connection failed
```

Explicação do Erro: O problema identificado estava relacionado à rede de internet utilizada durante a comunicação entre o Arduino e a API. A conexão da rede original estava bloqueando a comunicação do dispositivo com a API, o que impossibilitava o funcionamento correto do sistema. Durante aproximadamente duas horas, a equipe de desenvolvimento realizou testes e análises para identificar a origem da falha. Após a investigação, constatou-se que a solução consistia apenas na troca para a rede alternativa previamente disponibilizada pelo professor, permitindo assim a comunicação adequada entre o Arduino e a API.

3.5.6 Perguntas

O que acontece se o ESP enviar dados incorretos?

R: Se o ESP enviar dados incorretos, a API deve rejeitar a informação, registrar o erro em log e não salvar no banco, garantindo que o sistema continue confiável e sem dados inválidos.

O sistema deve aceitar qualquer informação?

R: Não. O sistema deve aceitar apenas dados válidos, rejeitando informações incorretas para garantir a confiabilidade e integridade dos dados.

Como sistemas industriais tratam falhas?

R: Sistemas industriais tratam falhas com validação de dados, redundância, monitoramento contínuo e geração de alertas, garantindo segurança e continuidade da operação.

3.6 INTEGRAÇÃO COM BANCO DE DADOS

```
-----
Gravado no MySQL:      ✖ Falha
Gravado no SQL Server: ✔ Sucesso
Gravado no Google Sheets: ✔ Sucesso
Data/Hora:              22/05/2026 11:56:53
-----
Temperatura:           23.2°C
Umidade:               56.0%
Arduino:               ID 1 (C4:D8:D5:2C:F4:4D)
Local:                 Laboratorio SENAI
-----
```

	id_leitura	id_dispositivo	temperatura	umidade	data_hora
1	15	1	24.30	58.00	2026-05-22 11:25:34.587
2	16	1	24.30	58.00	2026-05-22 11:25:35.363
3	17	1	24.40	57.00	2026-05-22 11:25:46.690
4	18	1	24.40	57.00	2026-05-22 11:25:52.243
5	19	1	24.40	57.00	2026-05-22 11:25:57.767
6	20	1	24.40	58.00	2026-05-22 11:26:03.323
7	21	1	24.40	57.00	2026-05-22 11:26:08.883
8	22	1	23.60	57.00	2026-05-22 11:33:10.207
9	23	1	23.60	57.00	2026-05-22 11:33:14.993
10	24	1	23.60	57.00	2026-05-22 11:33:24.923
11	25	1	23.70	57.00	2026-05-22 11:33:36.317
12	26	1	23.70	57.00	2026-05-22 11:33:46.043
13	27	1	23.70	57.00	2026-05-22 11:33:55.863
14	28	1	23.70	56.00	2026-05-22 11:34:05.777
15	29	1	23.80	55.00	2026-05-22 11:45:22.400
16	30	1	23.70	55.00	2026-05-22 11:45:32.333
17	31	1	23.70	57.00	2026-05-22 11:45:41.947
18	32	1	23.70	57.00	2026-05-22 11:45:51.590
19	33	1	23.30	57.00	2026-05-22 11:51:01.233

```

@app.post("/dados")
async def receber_dados(payload: ESPPayload): #recebe os dados do arduino
    # Analisa a temperatura e define o status
    temp = payload.sensores.temperatura
    if temp >= 21 and temp <= 28:
        status = "✅ | Normal"
    elif temp < 20:
        status = "❄️ | Frio"
    elif temp > 28 and temp <= 35:
        status = "🔥 | Quente"
    else:
        status = "⚠️ | ALERTA: Temperatura Extrema!"

    # Imprime no console para sabermos que chegou
    print("\n--- Novo dado recebido ---")
    print(f"📡 Dispositivo: {payload.device}")
    print("-" * 30)
    print(f"🌡️ Temperatura: {payload.sensores.temperatura}°C | Status: {status}")
    print(f"💧 Umidade: {payload.sensores.umidade}%")
    print(f"🔘 Botões: B1 {payload.entradas.botao1} | B2 {payload.entradas.botao2}")
    print(f"📶 Sinal WiFi: {payload.wifi_rssi} dBm")
    print("-" * 30)

```

- **Como a API se conecta ao banco:**

Para conectar a API ao banco de dados, foram definidos os principais dados de acesso, como usuário (User), senha, Host e o nome do banco.

Essas informações permitem que a aplicação se conecte corretamente ao banco e armazene os dados recebidos pelo sistema de forma organizada.

- **Como os dados são inseridos:**

Os dados recebidos pela API são inseridos diretamente no banco de dados utilizando comandos INSERT no próprio código Python. Esse processo permite que as informações coletadas pelo sistema sejam armazenadas de forma automática e organizada, garantindo que cada registro seja salvo corretamente no banco.

- **Como ocorre a organização das informações:**

A organização dos dados é feita de forma estruturada, onde cada informação é separada e classificada em seu respectivo campo. Isso garante que os dados fiquem bem organizados e de fácil acesso dentro do sistema.

3.7 IMPLEMENTAÇÃO DA INTEGRAÇÃO COM GOOGLE SHEETS

A implementação da integração com o Google Sheets tem como objetivo permitir que os dados coletados pelo ESP8266 sejam enviados automaticamente para uma planilha na nuvem, facilitando a visualização, análise e compartilhamento das informações em tempo real.

Para isso, a equipe deve inicialmente escolher a forma de integração entre a API e o Google Sheets. Entre as opções disponíveis estão o uso da API oficial do Google Sheets, bibliotecas em Python como o `gsread` ou serviços intermediários. A alternativa mais recomendada é utilizar a API oficial em conjunto com bibliotecas Python, pois oferece maior segurança e automação no processo.

Após definir a forma de integração, é necessário implementar o envio dos dados. A API deve ser programada para enviar automaticamente as informações recebidas para a planilha, adicionando novos registros a cada envio. Esses dados devem incluir informações como data

e hora, temperatura, umidade, rotação e estado do sistema.

Em seguida, é fundamental validar a sincronização dos dados. O sistema deve verificar se as informações foram realmente inseridas na planilha. Isso pode ser feito por meio da análise da resposta da API do Google, conferência direta na planilha ou testes realizados pela equipe. Caso o envio seja bem-sucedido, o sistema deve registrar essa operação como concluída com sucesso.

Por fim, é necessário implementar o registro de erros de integração. Se ocorrer alguma falha, como problemas de conexão ou autenticação, o sistema deve registrar essas ocorrências em logs para análise posterior. Além disso, os dados não devem ser perdidos, devendo permanecer armazenados no banco de dados até que a integração seja restabelecida.

Dessa forma, a integração com o Google Sheets garante maior acessibilidade e organização dos dados, ao mesmo tempo em que mantém a confiabilidade do sistema.

3.7.1 Autenticação Definida

Autenticação é o processo que as empresas usam para confirmar que apenas as pessoas, serviços e aplicativos certos com as permissões certas podem obter recursos organizacionais. É uma parte importante da segurança cibernética, pois a prioridade número um de um invasor é obter acesso não autorizado aos sistemas. Eles fazem isso roubando o nome e as senhas dos usuários que têm acesso. O processo de autenticação inclui três etapas principais:

- **Identificação:** normalmente, os usuários estabelecem quem são por meio de um nome de usuário.
- **Autenticação:** normalmente, os usuários provam que são quem dizem ser digitando uma senha (algo que apenas o usuário deve saber), mas para fortalecer a segurança, muitas organizações também exigem que provem sua identidade com algo que possuem (um telefone ou dispositivo de token) ou algum identificador físico (impressão digital ou leitura facial).
- **Autorização:** o sistema verifica se os usuários têm permissão para o sistema que estão tentando acessar.

3.7.2 Perguntas

- **O Google Sheets pode substituir um banco de dados?**

R: Não, o Google Sheets não substitui um banco de dados, pois é limitado, mas pode ser usado para visualização e relatórios.

- **Qual o papel da nuvem nesse projeto?**

R: A nuvem permite armazenar, acessar e compartilhar os dados do sistema pela internet, possibilitando monitoramento em tempo real de qualquer lugar, além de integrar serviços como o Google Sheets.

- **O que acontece se a integração falhar?**

R: Se a integração falhar, os dados não são enviados para o Google Sheets, mas devem continuar sendo armazenados no banco de dados, enquanto o sistema registra o erro em log para a correção posterior, garantindo que nenhuma informação importante seja perdida.

	A	B	C	D	E
1	Data/Hora	Dispositivo	Temperatura (°C)	Umidade (%)	Status
2	14/05/2026 16:17:10	ESP8266_M302	20	40	normal
3	14/05/2026 16:17:21	ESP8266_M302	20,1	40	normal
4	14/05/2026 16:17:31	ESP8266_M302	20,1	40	normal
5	14/05/2026 16:17:42	ESP8266_M302	20,1	41	normal
6	14/05/2026 16:17:52	ESP8266_M302	20,1	41	normal
7	14/05/2026 16:18:02	ESP8266_M302	20,1	40	normal
8	14/05/2026 16:18:13	ESP8266_M302	20,1	40	normal
9	14/05/2026 16:18:23	ESP8266_M302	20,2	40	normal
10	14/05/2026 16:18:33	ESP8266_M302	20,2	40	normal
11	21/05/2026 11:14:12	F4:CF:A2:61:34:F	24,7	55	normal
12	21/05/2026 11:14:23	F4:CF:A2:61:34:F	24,6	54	normal
13	21/05/2026 11:14:34	F4:CF:A2:61:34:F	24,6	54	normal
14	21/05/2026 11:14:45	F4:CF:A2:61:34:F	24,6	53	normal
15	21/05/2026 11:14:55	F4:CF:A2:61:34:F	24,6	54	normal
16	21/05/2026 11:15:06	F4:CF:A2:61:34:F	24,6	53	normal
17	21/05/2026 11:15:16	F4:CF:A2:61:34:F	24,5	53	normal
18	21/05/2026 11:15:27	F4:CF:A2:61:34:F	24,6	53	normal
19	21/05/2026 11:15:37	F4:CF:A2:61:34:F	24,5	54	normal
20	21/05/2026 11:15:48	F4:CF:A2:61:34:F	24,5	55	normal
21	21/05/2026 11:15:58	F4:CF:A2:61:34:F	24,4	54	normal
22	21/05/2026 11:16:08	F4:CF:A2:61:34:F	24,3	53	normal
23	21/05/2026 11:16:19	F4:CF:A2:61:34:F	24,3	54	normal
24	21/05/2026 11:16:29	F4:CF:A2:61:34:F	24,2	54	normal
25	21/05/2026 11:16:39	F4:CF:A2:61:34:F	24,2	55	normal

3.8 SEGURANÇA INICIAL DO SISTEMA

3.8.1 Pesquisa

- **Tokens simples:**

Tokens simples são pequenas partes de informação usadas por sistemas e APIs para identificar, autenticar ou permitir acesso a um serviço. De forma simples, um token funciona como uma “chave digital” temporária. Quando um dispositivo ou sistema, como um Arduino, quer se conectar a uma API, ele envia esse token para provar que tem permissão de acesso.

- **Validação de requisições:** A validação de requisições é o processo de verificar se as informações enviadas para um sistema ou API estão corretas, completas e autorizadas antes de serem processadas. Esse procedimento garante maior segurança e evita erros de comunicação entre dispositivos e sistemas. Durante a validação, podem ser conferidos dados como tokens de acesso, formato das informações enviadas e permissões da requisição. Caso a requisição não atenda aos critérios definidos, ela é bloqueada ou rejeitada pelo sistema.

- **Controle básico de acesso:** O controle básico de acesso é um mecanismo utilizado para limitar e gerenciar quem pode acessar determinado sistema, dispositivo ou API. Esse controle funciona por meio da verificação de informações como usuário, senha ou token de autenticação, garantindo que apenas usuários ou dispositivos autorizados possam realizar conexões e operações no sistema. O objetivo principal é aumentar a segurança, proteger dados e evitar acessos não autorizados.

- **Organização segura das credenciais:** A organização segura das credenciais consiste em armazenar e gerenciar informações sensíveis, como senhas, tokens e chaves de acesso, de forma protegida e estruturada. Essa prática reduz riscos de vazamento de dados e acessos não autorizados, além de facilitar a manutenção e o gerenciamento do sistema. Entre as principais medidas estão evitar expor credenciais diretamente no código, utilizar arquivos de configuração seguros e restringir o acesso apenas a usuários autorizados.

3.8.2 Perguntas

- **Por que um sistema IoT precisa de segurança?**

R: Um sistema IoT precisa de segurança para proteger os dados, evitar acessos não autorizados e impedir ataques que podem comprometer o funcionamento do sistema. Sem segurança, invasores podem manipular informações, causar falhas na automação e até interromper todo sistema.

- **O que pode acontecer se a API ficar exposta?**

R: O sistema fica vulnerável a invasões e pode ter seu funcionamento comprometido.

- **Como ataques podem impactar a automação?**

R: Ataques podem tornar a automação insegura, instável e não confiável.

CONCLUSÃO

Nesta etapa, o projeto evoluiu de um sistema embarcado simples para uma solução integrada de automação, capaz de receber, processar, validar e armazenar informações automaticamente. A implementação da API permitiu a comunicação eficiente entre o ESP8266, o banco de dados e o Google Sheets, garantindo maior organização e monitoramento dos dados coletados.

Além disso, foram aplicados mecanismos básicos de segurança, validação de requisições e registro de logs, aumentando a confiabilidade do sistema. Durante os testes, a equipe identificou e corrigiu falhas de integração e comunicação, aprimorando a estabilidade da aplicação.

Com isso, o projeto passou a demonstrar na prática conceitos de IoT, integração de sistemas, automação, armazenamento de dados e comunicação em rede, proporcionando experiência no desenvolvimento de soluções conectadas e escaláveis.

REFERÊNCIAS

<https://learn.microsoft.com/pt-br/viva/goals/gsheets-integration> Acessado dia 14 de Maio de 2026.

<https://www.microsoft.com/pt-br/security/business/security-101/what-is-authentication> Acessado dia 14 de Maio de 2026.

<https://www.ibm.com/br-pt/think/topics/rest-apis> Acessado dia 14 de Maio de 2026.

<https://developer.mozilla.org/pt-BR/docs/Web/HTTP/Reference/Methods> Acessado dia 14 de Maio de 2026.

<https://www.ibm.com/docs/pt-br/cics-ts/5.6.0?topic=programs-clientserver-model> Acessado dia 14 de Maio de 2026.

<https://www.dio.me/articles/o-que-sao-endpoints-e-rotas-de-uma-api> Acessado dia 14 de Maio de 2026.